```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
builder.Services.AddRazorPages()
    .AddRazorPagesOptions(opts =>                                    ❶
    {
        opts.Conventions.Add(                                        ❷
            new PageRouteTransformerConvention(                      ❷
                new KebabCaseParameterTransformer()));               ❷
        opts.Conventions.AddPageRoute(                               ❸
            "/Search/Products/StartSearch", "/search-products");     ❸
    });

WebApplication app = builder.Build();

app.MapRazorPages();

app.Run();
```

❶ AddRazorPagesOptions can be used to customize the conventions used by Razor Pages

❷ Registers the parameter transformer as a convention used by all Razor Pages

❸ AddPageRoute adds a route template to Pages/Search/Products/StartSearch.cshtml.

The `AddPageRoute()` convention adds an alternative way to execute a single Razor Page. Unlike when you customize the route template for a Razor Page using the `@page` directive, using `AddPageRoute()` adds an extra route template to the page instead of replacing the default. That means there are two route templates that can access the page.

TIP Even the name of the Pages root folder is a convention that you can customize! You can customize it by setting the `RootDirectory` property inside the `AddRazorPageOptions()` configuration lambda.

If you want even more control of your Razor Pages route templates, you can implement a custom convention by implementing the `IPageRouteModelConvention` interface and registering it as a custom convention. `IPageRouteModelConvention` is one of